

OneSearch: A Preliminary Exploration of the Unified End-to-End Generative Framework for E-commerce Search

Ben Chen^{*†}, Xian Guo^{*}, Siyuan Wang^{*}, Zihan Liang^{*}, Yue Lv, Yufei Ma, Xinlong Xiao, Bowen Xue, Xuxin Zhang, Ying Yang, Huangyu Dai, Xing Xu, Tong Zhao, Mingcan Peng, Xiaoyang Zheng, Chao Wang, Qihang Zhao, Zhixin Zhai, Yang Zhao, Bochao Liu, Jingshan Lv, Jing Chen, Xiao Liang, Yuqing Ding, Chenyi Lei, Wenwu Ou, Han Li, Kun Gai
Kuaishou Technology, Beijing, China
Contact: {chenben03@kuaishou.com}

Abstract

Traditional e-commerce search systems employ multi-stage cascading architectures (MCA) that progressively filter items through recall, pre-ranking, and ranking stages. While effective at balancing computational efficiency with business conversion, these systems suffer from fragmented computation and optimization objective collisions across stages, which ultimately limit their performance ceiling. To address these, we propose **OneSearch**, the first industrial-deployed end-to-end generative framework for e-commerce search. This framework introduces three key innovations: (1) a Keyword-enhanced Hierarchical Quantization Encoding (KHQE) module, to preserve both hierarchical semantics and distinctive item attributes while maintaining strong query-item relevance constraints; (2) a multi-view user behavior sequence injection strategy that constructs behavior-driven user IDs and incorporates both explicit short-term and implicit long-term sequences to model user preferences comprehensively; and (3) a Preference-Aware Reward System (PARS) featuring multi-stage supervised fine-tuning and adaptive reward-weighted ranking to capture fine-grained user preferences. Extensive offline evaluations on large-scale industry datasets demonstrate OneSearch’s superior performance for high-quality recall and ranking. The rigorous online A/B tests confirm its ability to enhance relevance in the same exposure position, achieving statistically significant improvements: +1.67% item CTR, +2.40% buyer, and +3.22% order volume. Furthermore, OneSearch reduces operational expenditure by 75.40% and improves Model FLOPs Utilization from 3.26% to 27.32%. The system has been successfully deployed across multiple search scenarios in Kuaishou, serving millions of users, generating tens of millions of PVs daily.

Keywords

E-commerce Search, End-to-End Generative Retrieval, Hierarchical Semantic Encoding, Reward System

^{*}Equal Contribution.

[†] Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference’17, Washington, DC, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

ACM Reference Format:

Ben Chen^{*†}, Xian Guo^{*}, Siyuan Wang^{*}, Zihan Liang^{*}, Yue Lv, Yufei Ma, Xinlong Xiao, Bowen Xue, Xuxin Zhang, Ying Yang, Huangyu Dai, Xing Xu, Tong Zhao, Mingcan Peng, Xiaoyang Zheng, Chao Wang, Qihang Zhao, Zhixin Zhai, Yang Zhao, Bochao Liu, Jingshan Lv, Jing Chen, Xiao Liang, Yuqing Ding, Chenyi Lei, Wenwu Ou, Han Li, Kun Gai. 2025. OneSearch: A Preliminary Exploration of the Unified End-to-End Generative Framework for E-commerce Search. In . ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

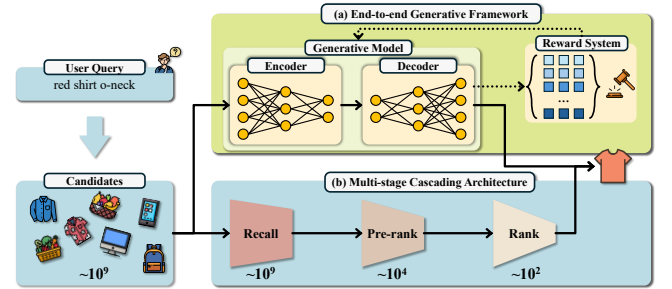


Figure 1: (a) Our proposed End-to-End generative retrieval framework, (b) the traditional multi-stage cascading architecture in E-commerce search.

1 Introduction

E-commerce search aims to retrieve items that align with the user’s real intention based on their search terms, behavioral preferences, and the available inventory. To enhance user experience, these systems are typically required to identify items that satisfy both semantic and personalized criteria from hundreds of millions of candidates within one second. Consequently, traditional search systems frequently employ the Multi-stage Cascading Architecture (MCA). As depicted in Figure 1(b), MCA adheres to a coarse-to-fine paradigm, wherein a query progresses through recall, pre-ranking, and ranking stages, ultimately returning the top-selected items. The recall stage operates on the entire set of item candidates ($\sim 10^9$), the pre-ranking stage narrows down the selection ($\sim 10^4$), and the final ranking stage evaluates only the top candidates ($\sim 10^2$).

MCA effectively balances the trade-off between system response time and sorting accuracy by progressively narrowing the pool of items at each stage. However, MCA inherently suffers from severe fragmented compute and objective collision issues [5, 6, 37]. Fragmented compute means most serving resources are allocated to



Figure 2: The main search entries in the Kuaishou Platform.

communication and storage rather than numerical computation. Regarding objective collision, MCA potentially employs multiple strategies to meet accuracy and diversity requirements. For instance, the recall and pre-ranking stages use lightweight models that tend to retrieve all relevant items, while the ranking stage implements complex reasoning of user preferences by introducing user historical sequences, query and item statistical features. The potential discarding of intended items due to multi-layer funnel filtering, along with heterogeneous optimization objectives, reducing the performance ceiling of search systems. Furthermore, traditional MCA lacks an understanding of cold-start queries and items, resulting in limited performance in long-tail sessions [3, 12].

In recent years, numerous efforts have been made to address the aforementioned issues. Some works focus on intra-stage optimization, such as EBR[14] for recall, DCN[36] and DSSM[15] for pre-ranking, DIN[49] and DeepFM[10] for the final ranking stage. Particularly, with the advancements in large language models (LLMs), a significant portion of research has emerged aiming to use generative models to tackle the challenges of each stage [4, 21, 23, 42, 51]. Another line of work focuses on resolving the inconsistency of objectives across the full stages [8, 43, 44], striving to ensure the effective transmission of intent items through sample construction and loss design. However, these approaches still struggle to overcome the inherent limitations of MCA. For instance, (pre-)ranking stages can only process top-k items retrieved from the previous stage. If an effective item that aligns with the user’s true intent is filtered out in an earlier stage, no matter how precise the subsequent models are, they cannot present this item to user.

In the past two years, a novel generative retrieval (GR) paradigm has emerged, which transforms the basic matching-based framework of MCA into generation-based approaches, addressing its inherent limitations [5, 12, 27, 31, 37, 45, 46]. This paradigm eliminates the need for multi-stage filtering by directly inputting query or user sequence information and outputting corresponding item candidates. Tiger[31] pioneered the development of end-to-end generative retrieval models for sequential recommendation, introducing the semantic IDs (SID) derived from each item’s content information for efficient item representation. LC-REC[45] proposed adapting LLMs by integrating collaborative semantics for recommendation, utilizing a series of specially designed tuning tasks. Subsequently, OneRec[5] was first implemented in a real industrial scenario, followed by OneSug[12] for query suggestion in e-commerce search, EGA[46] for advertising, and OneLoc[37] for local life services.

While for e-commerce search, these methods are not so effective due to several unique and critical challenges: 1) Item information, such as titles, keywords, and detail pages, tends to be lengthy with redundant irrelevant noise, as sellers often add unrelated keywords to increase exposure. Moreover, the semantic order in item descriptions is weak. Essential information such as brand names, attribute words, and categories often appears in the text without regard for position [21]. This lack of global coherence can severely mislead representation models, leading to incorrect judgments. 2) Unlike recommendations, there is a strong relevance constraint between search queries and items. Queries typically consist of 2-3 short keywords, and any mismatch in attributes can result in significant relevance issues. Although semantic ID-based GR models can construct hierarchical, learnable representations of items, they inevitably lead to the loss of core attribute representations, as they tend to learn shared information under the same SIDs. 3) Uncovering the users’ latent search intents is also a core challenge. When users enter concise queries or search for a completely new category, it is crucial to effectively combine query content with user behavior profiles to infer the user’s true search intent.

To address these challenges, we propose **OneSearch**, an end-to-end generative framework for e-commerce search, which includes:

1) **Keyword-enhanced Hierarchical Quantization Encoding** module. KHQE employs a keyword-enhanced semantic collaborative encoder to highlight the core attributes of items. It then uses RQ-Kmeans for hierarchical feature encoding and OPQ for unique features quantization of each item. This encoding effectively reduces interference from redundant irrelevant noise, thereby enhancing the relevance between queries and generated items.

2) **Multi-view User Behavior Sequence (Mu-Seq) injection** strategy. This strategy introduces a weighted decay click behavior sequence into the user ID to construct a distinctive user representation, then explicitly incorporates short behavior sequences in prompts to learn recent user preferences and implicitly includes long behavior sequences to model the user profile, achieving multi-view modeling of user personalized behavior.

3) **Preference Aware Reward System (PARS)**. We design a multi-stage supervised fine-tuning process for semantic alignment and personalization, followed by an adaptive reward system that leverages hierarchical user behavior signals and list-wise preference optimization. This system enables the model to learn preference differences among items while maintaining query-item relevance constraints through hybrid ranking that combines reward model guidance with direct user interaction feedback.

Extensive offline evaluations are conducted on real user search logs, and the significant performance boosts demonstrate OneSearch’s effectiveness for e-commerce search. We also make multiple strict A/B testings in the Kuaishou mall search platform to verify its online effectiveness. The pure OneSearch can get comparable performance to the online MCA. By the introduction of RQ-OPQ and long behavior sequence, OneSearch can confidently improve item CTR by 1.45%, PV CTR by 1.40%. While applying the additional reward model selection can yield a 1.102% increase in search pvs, 1.67% in item CTR, 3.14% in PV CTR, 1.78% in PV CVR, 2.40% in buyers volume, and 3.22% in order volume. As a side note, we further performed an MCA testings that only included recall, pre-ranking stage, while item CTR dropped by 9.971% and order

by 39.144%. These results effectively demonstrate the performance of OneSearch. Finally, the pure OneSearch can save about 75.40% operational expenditure and improve the Model Flops Utilization (MFU) from 3.26% of MCA to 27.32%. It has been successfully deployed for the entire traffic on the detail page search, 50% traffic on the mall search, and 20% traffic on the homepage search platform, for further investigation. To the best of our knowledge, it is the first industrial-deployed end-to-end generative framework for e-commerce search. We hope that exploration could further pave the way for smarter GRs in Search.

The main contributions of this work are summarized as follows:

- We propose a novel keyword-enhanced hierarchical quantization encoding, which can generate semantic IDs balanced with context features and collaborative signals for queries/items. The keyword enhancement strategy can further reduce the Interference from irrelevant noise, and strengthen the relevance constraints of GRs.
- We devise a multi-view user behavior sequential injection strategy, with introducing sequence into the user ID representation, and the input prompt explicitly and implicitly. This approach can facilitate GRs' reasoning about user profiles and preferences.
- We design a preference aware reward system, which contains a multi-stage SFT process, as well as an adaptive reward model, to improve the model's personalized ranking capability.
- Finally, we presented OneSearch, the first industrial-deployed end2end generative framework for e-commerce search. Various offline and online A/B tests are conducted, verifying its effectiveness and efficiency for the real e-commerce search scenarios.

2 Related Works

2.1 Generative Retrieval and Recommendation

In recent years, Generative Retrieval (GR) has garnered significant attention from both academia and industry due to its remarkable performance. This emerging retrieval paradigm, which regards large-scale retrieval as sequence-to-sequence generation tasks, has outperformed traditional ANN-based models such as EBR [14] and RocketQA [30], spurring increased exploration in the fields of search and recommendation. Notable contributions in this area include Tiger [31], DSI [33], and LC-REC [45]. Tiger [31] pioneered the development of end-to-end generative retrieval models for sequential recommendation, introducing semantic IDs (SID) derived from each item's content information for efficient item representation. LC-REC [45] proposed adapting large language models (LLMs) by integrating collaborative semantics for recommendation, utilizing a series of specially designed tuning tasks.

Most GR models serve merely as supplementary recall sources within online systems, thereby overlooking these models' inherent rich semantic and powerful reasoning abilities for potential use in (pre-)ranking stages. In the area of video recommendation, OneRec [5] was the first to unify recall, pre-ranking, and ranking within a single generative model. This was achieved with the assistance of session-wise generation and iterative preference alignment, resulting in substantial improvements in practical online metrics. EGA [46] represents a significant departure from both traditional multi-stage cascading architectures (MCA) and existing generative retrieval models by introducing a unified framework that holistically models the entire advertising pipeline. UniROM [29] employs

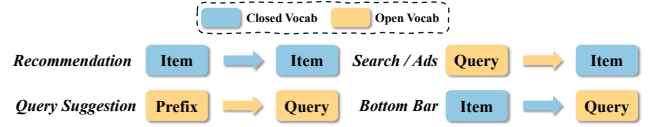


Figure 3: The input and output differences among Recommendation, Search/Ads, Query Sug and Bottom Bar.

a hybrid feature service to efficiently decouple user and advertising features, and RecFormer [20], a variation of Transformer, captures both intra- and cross-sequence interactions.

2.2 Generative Retrieval for Search

These two years, most advancements in generative retrieval have been focused on recommendations. This is because search systems face three major challenges: 1) multiple and low-density item information, 2) strong relevance constraints between search queries and items, and 3) inference barriers to users' potential search intentions. Consequently, the current traditional e-commerce search systems still adopt a multi-stage cascading architecture. However, some efforts have been made to optimize current search systems using generative retrieval (GR).

The first example is GenR-PO [22], which utilizes multi-span identifiers to represent raw item titles. This approach transforms the task of generating titles from queries into the task of generating multi-span identifiers from queries, thereby simplifying the generation process. Subsequently, a constrained search method is employed to identify key spans for retrieving the final item, which has proven beneficial for online recall systems. Another notable example is the Generative Retrieval and Alignment Model (GRAM) [27], which performs joint training on text information from both queries and products to generate shared text identifier codes. GRAM employs a co-alignment strategy to optimize these codes for maximizing retrieval efficiency and is deployed on the JD search engine to enhance both the recall and pre-ranking stages.

Recently, we proposed OneSug [12], which incorporates a prefixQuery representation enhancement module to enrich prefixes using semantically and interactively related queries to bridge content and business characteristics, an encoder-decoder generative model that unifies the query suggestion process, and a reward-weighted ranking strategy with behavior-level weights to capture fine-grained user preferences. It is the first end-to-end generative framework for e-commerce query suggestion, and has been verified to have substantial improvements in user clicks and conversion.

These GR methods demonstrate appealing performance in the realm of search, recommendation, bottom navigation, advertising, and even query suggestion. They are not suitable for e-commerce search. As illustrated in Figure 3, the inputs and outputs of recommendation are the closed-vocabulary items or videos, thus the pure semantic ID tokenization is suitable for its diverse item generation. The inputs and outputs of query suggestion are the full open-vocabulary textual descriptions, so that it can directly use the transformer architecture. For the bottom bar and search engine, either the inputs or the outputs are open-vocabulary, which represents a significant departure from both OneRec and EGA.

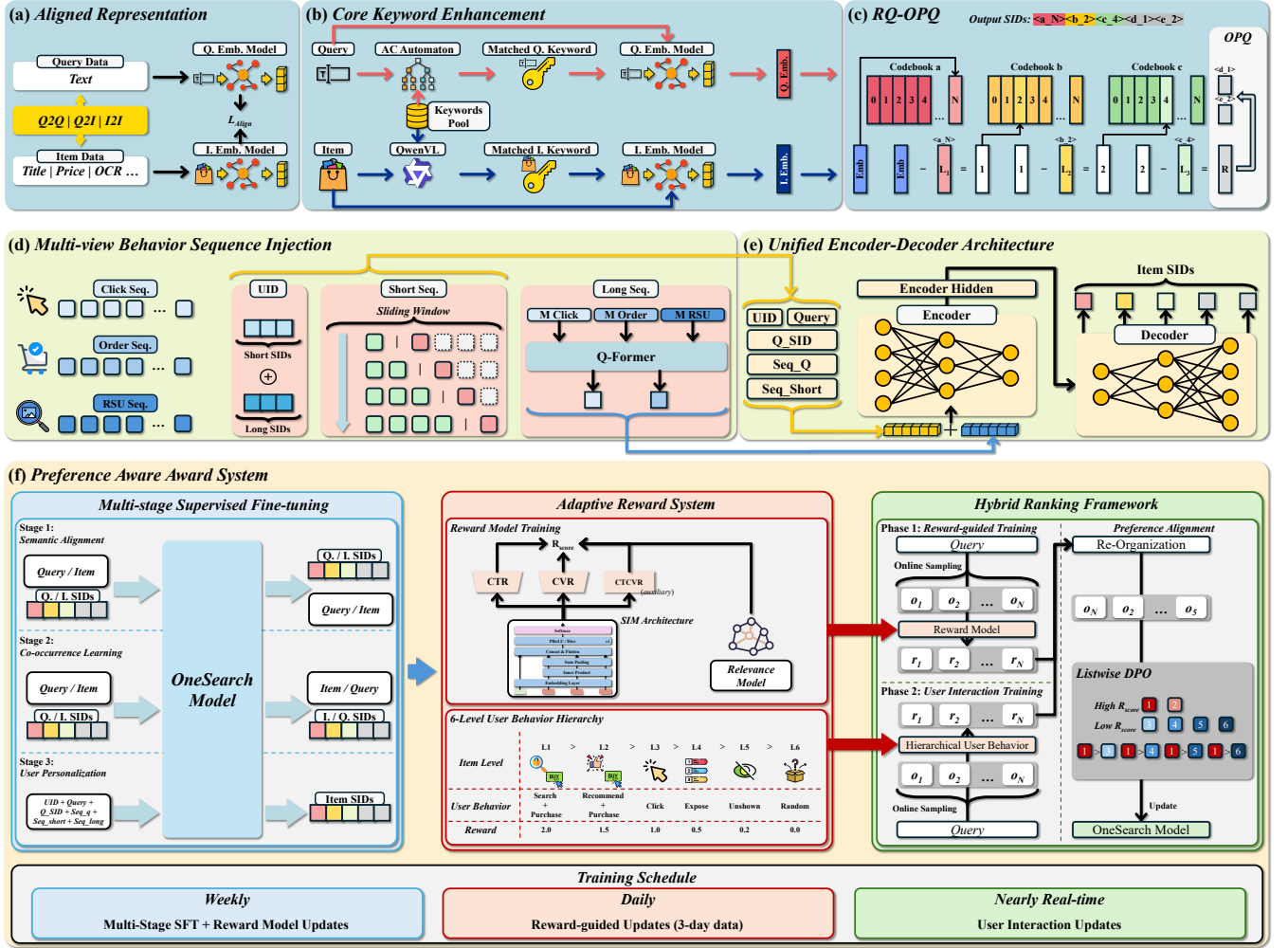


Figure 4: The Framework of OneSearch includes four parts: 1) Keyword-enhanced Hierarchical Quantization Encoding, which adopts aligned representation and core keyword to construct an elaborative hierarchical quantization tokenization schema; 2) Multi-view Behavior Sequence Injection, utilizing sequences to reason the user profiles and preferences; 3) Unified encoder-decoder Architecture that integrates produced features for generative retrieval; and 4) Preference aware reward system, which contains a multi-stage SFT procedure and an adaptive reward system, to enhance the model’s personalized ranking capability.

3 Methodology

In this section, we detail the proposed OneSearch, an end-to-end framework for e-commerce search, in four parts. We elaborate on the keyword-enhanced hierarchical quantization encoding in § 3.1, then we introduce the strategy of multi-view user behavior sequence injection in § 3.2 and unify the encoder-decoder architecture in § 3.3. In § 3.4, a preference aware reward system is proposed, which includes a multi-stage supervised finetuning process, as well as an adaptive reward system for personalized ranking learning. The framework of OneSearch is illustrated in Figure 4.

3.1 Hierarchical Quantization Encoding

Encoding items into Semantic IDs (SIDs) is crucial for the success of generative retrieval models. This process converts continuous

semantic representations into discrete ID sequences using a coarse-to-fine quantization approach, ensuring that items with the same SID share the same information [5, 16, 31]. However, common quantization methods (e.g., VQ-VAE [34], RQ-VAE [18], or RQ K-means [24]) tend to tokenize shared signals among items using a reduced and fixed vocabulary, which results in the loss of distinctive attributes for each item. This information loss causes many similar but not identical items sharing the same SIDs, while lower codebook utilization and independent coding rate limit the performance potential of GRs.

Some other GRs attempt to use finite scalar quantization (FSQ) [26] or optimized product quantization (OPQ) [9] to tokenize as much effective information as possible [13]. However, these methods fail to represent core attributes among similar items hierarchically. Therefore, we propose to combine both encoding paradigms.

Table 1: 18 structured attributes using Named Entity Recognition in the KuaiShou e-commerce search platform.

Entity	Modifier	Brand	Material	Style	Function
Location	Audience	Color	Marketing	Season	Pattern
Scene	Specifications	Price	Model	Anchor	Series

First, we leverage domain knowledge to extract core attributes of queries and items, to enhance the learned semantic and collaborative representations. Then, we use RQ-Kmeans for hierarchical feature encoding and OPQ for quantizing the unique features of each item. This encoding method effectively reduces interference from redundant irrelevant noise, thereby enhancing the relevance between queries and generated items.

3.1.1 Aligned collaborative and semantic representation. We integrate semantic knowledge with collaborative signals by aligning the representations of historically interactive query-item pairs. Firstly, we select high-quality query2query, item2item, and query2items pairs from real user search logs using existing retrieval models like ItemCF [32] and Swing [41]. Then we collect the content information like query text, item title, item price, keywords, OCR (image-to-text), as well as the statistical business characteristics, such as the number of clicks, add-to-cart, and purchases during a certain time. All these features are processed with a distilled BGE [38] to generate a content embedding for each query e_q and item e_i . Finally, we filter all data with a cosine similarity larger than 0.6 to ensure all pairs are content-relevant.

We design four types of interrelated tasks to align collaborative and semantic representation: 1) the query2query and item2item contrastive loss $\mathcal{L}_{q2q}$, $\mathcal{L}_{i2i}$ to align representations of collaboratively similar pairs, 2) a query2item contrastive loss $\mathcal{L}_{q2i}$ to ensure that BGE can reflect real business characteristics, 3) a query2item margin loss $\mathcal{L}_{rank}$ to further learn the collaborative signal deviation of query-item pairs with different behavior levels (like show, click, order), 4) a hard sample relevance correction loss $\mathcal{L}_{rel}$. For pairs with a threshold similarity, we use LLM to score the relevance based on full context information of query and item, and then let the distilled BGE model fit this score.

Then we train the aligned model with the total loss as:

$$\mathcal{L}_{align} = \lambda_1 \cdot \mathcal{L}_{q2q} + \lambda_2 \cdot \mathcal{L}_{i2i} + \lambda_3 \cdot \mathcal{L}_{q2i} + \lambda_4 \cdot \mathcal{L}_{rank} + \lambda_5 \cdot \mathcal{L}_{rel} \quad (1)$$

where λ_i is an adjustable parameter for different objectives.

3.1.2 Core Keyword Enhancement. Item textual information often contains redundancy with many irrelevant words and even mutually exclusive attributes. While these attributes can increase item exposure, the weak semantic order caused by numerous stacked attributes makes it difficult for encoders to model key information. Here we propose using core keyword features to enhance textual representation, thereby obtaining semantic IDs dominated by these keywords. Specifically, we identified 18 structured attributes using Named Entity Recognition (NER) and mined click query-item pairs from the past 1 year as labeled data. We then compiled a list of keywords for each attribute, ranked by page views (PV) in descending order, and selected high-frequency keywords as core ones. We

Table 2: The codebook utilization rate (CUR) and independent coding rate (ICR) for various RQ-Kmeans configurations. The last + means balanced operation for all levels.

Configurations	CUR_{L1}	CUR_{L1*L2}	CUR_{Total}	ICR
1024-1024-1024	100%	54.27%	1.72%	36.67%
\+keywords	100%	65.40%	2.03%	40.25%
2048-1024-512	100%	46.88%	1.98%	37.80%
\+keywords	100%	57.16%	2.51%	40.76%
4096-1024-256	99.90%	39.21%	2.27%	36.98%
\+keywords	100%	48.95%	2.94%	40.52%
\+l3 balanced	100%	48.95%	10.31%	60.01%
4096-1024-512	99.90%	39.21%	1.30%	40.54%
\+keywords	100%	48.95%	1.64%	43.32%
\+l3 balanced	100%	48.95%	7.03%	68.08%
4096-1024-512+	99.93%	41.45%	0.51%	33.47%

Table 3: Performance Comparisons of three Tokenization Schemas. Metrics are evaluated on the real click pairs.

Method	CUR_{Total}	ICR	Recall@10	MRR@10
OnlineMCA	-	-	0.3440	0.1323
RQ-VAE	1.17%	38.83%	0.2171	0.0689
RQ-Kmeans	7.03%	68.08%	0.2844	0.1038
RQ-OPQ	-	91.91%	0.3369	0.1194

employ Qwen-VL [2] as the discriminant model to identify corresponding keywords for each item, while for queries, we use the Aho-Corasick Automaton [1] for rapid matching during inference.

All these core keywords are input into the former trained model to obtain vectors e_k^i consistent with the item representation distribution. The final optimized representations for each query e_q^o and item e_i^o are given by:

$$e_q^o = \frac{1}{2}(e_q + \frac{1}{m} \sum_{i=1}^m e_k^i), \quad e_i^o = \frac{1}{2}(e_i + \frac{1}{n} \sum_{j=1}^n e_k^j). \quad (2)$$

This approach enhances the role of core keywords in encoding. As shown in Table 1, the core keyword enhancement scheme improves the codebook utilization rate (CUR) of RQ-Kmeans at each level and further increases the independent coding rate (ICR). For example, with a configuration of 4096-1024-512 (shown in Table 2), it results in a 0.10% CUR increment for Level 1, 24.84% for Level 2, and 26.15% for Level 3, as well as the overall ICR increasing by 6.86%.

3.1.3 RQ-OPQ Hierarchical Quantization Tokenization. Common SID tokenizers, such as RQ-VAE, VQ-VAE, and RQ K-means, focus on encoding shared features among similar items, which can result in the loss of distinctive features for each item, ultimately degrading the performance of generative retrieval models (GRs). Furthermore, RQ-VAE has shown weaker performance compared to RQ-Kmeans [5, 17], as validated in Table 3. Therefore, we adopt RQ-Kmeans as the foundational tokenizer.

We use the codebook utilization rate (CUR) and the independent coding rate (ICR) as evaluation metrics. The basic codebook size is set to 1024, and the number of codebook layers is set to 3, which aligns with the number of items in the candidate pool. However, e-commerce items have more varied categories and attributes, and RQ-Kmeans tends to prioritize clustering shared prominent features in the former layers. In order to make more concise tokenization, we maintain the capacity of RQ-Kmeans while increasing the codebook size of the former layer to ensure more comprehensive learning of prominent features. As depicted in Table 2, we tested three configurations: (1024,1024,1024), (2048,1024,512), and (4096,1024,256). The codebook size of 4096 achieves higher CUR and ICR, and the Core Keyword Enhancement scheme ($\backslash$ +keywords) shows further improvement. Considering that the search system should encode the entered query similarly and that merchants often increase the number of listed items during global shopping festivals (e.g., 11.11 and 6.18), we further expanded the codebook size to (4096-1024-512). We found that the semantic tokens increased by 11.56% (as $2 \cdot 1.64\%/2.94\% - 1$), and the independent coding rate increased to 43.42% compared to the (4096-1024-512) ($\backslash$ +keywords).

To further improve CUR and ICR, OneRec-V1 [5] proposed using full layers balanced k-means. However, for complex fine-grained attributes of items, forcing them into the same cluster in the early stages can lead to hierarchical clustering collapse. As shown in Table 2, the CUR_{total} for the balanced k-means operation on full layers (4096-1024-512⁺) is much lower than the ($\backslash$ +keywords) configuration. The CUR drastically decreased from 48.95% of CUR_{L1+L2} to 1.64% in CUR_{total} , indicating that many similar items were assigned the same ID. Therefore, we propose applying balanced k-means only to the codebook of the third layer to achieve independent encoding of similar items. As shown in ($\backslash$ +l3 balanced), the CUR_{Total} increased from 1.64% to 7.03%, while the ICR improved by 57.15%.

Although RQ-Kmeans can construct hierarchical, learnable SIDs for items, it inevitably discards the residual embedding computed after the last clustering. However, this residual embedding contains the distinctive attributes of each item. Therefore, we further use OPQ for quantizing the unique features. The RQ method handles hierarchical semantics, while PQ is adopted for lateral characteristics. This combined tokenizer can more comprehensively represent the fine-grained features of items, thereby enhancing the relevance constraints for GR models. As shown in Table 3, the two additional SIDs (256-256) generated by OPQ significantly improve the ICR metric and enhance the recall and ranking capabilities of GRs. More detailed testing is introduced in §4.2.

3.2 Multi-view Behavior Sequence Injection

We introduce the behavior sequence into GRs from three perspectives. First, we propose a behavior sequence constructed user ID scheme to achieve the distinctive user representation, then explicitly incorporates short behavior sequences in prompt text to learn recent user preferences and implicitly includes long behavior sequences to model user profile, achieving multi-view modeling of user personalized behavior.

3.2.1 Behavior Sequence Constructed User IDs. Tiger[31] prepends user-specific tokens into prompts to achieve unique user identification, which is randomly hashed and assigned to a fixed-size

vocabulary. However, this method has been shown to be ineffective. We believe that such random IDs do not adequately represent user personalization, as the fixed-size vocabulary may assign the same ID to users with different behaviors. Here, we propose a behavior sequence-constructed user ID scheme to achieve distinctive user representation. Formally, the short behavior sequence consists of the user's latest clicked items, denoted as $Seq_{short} = \{s_1, s_2, \dots, s_m\}$, where s_i is the i_{th} item the user clicked, and m is the total number of latest clicked items. The long behavior sequence is a list of ordered items arranged in chronological order, denoted as $Seq_{long} = \{l_1, l_2, \dots, l_n\}$. The user ID is then computed as the concatenation of SID_{short} and SID_{long} :

$$\begin{aligned} SID_{short} &= \lceil \sum_{i=s_1}^m \lambda_i \cdot SID_{s_i} \rceil, \quad \text{where } \lambda_i = \frac{\exp(\sqrt{i})}{\sum_i^m \exp(\sqrt{i})}, \\ SID_{long} &= \lceil \sum_{j=l_1}^n \mu_j \cdot SID_{l_j} \rceil, \quad \text{where } \mu_j = \frac{\exp(\sqrt{j})}{\sum_j^n \exp(\sqrt{j})}. \end{aligned} \quad (3)$$

So that the length of User IDs is 10. For new-coming or cold-start users, we count the most clicked items for each query based on query-item occurrence and sort them in reverse order by page views as default behavior sequences.

3.2.2 Explicit Short Behavior Sequence. The short (recent) and long behavior sequences are crucial features in modeling user preferences. The short behavior sequence primarily reflects recent user preferences, while the long behavior sequence represents a user's profile. For example, a soon-to-be-enrolled college student may recently purchase items related to a new dormitory or their major. In contrast, purchases made six months ago might have been more related to college entrance exams or stationery. Therefore, for a user's next search, we prioritize the recent behavioral information. However, the long behavior sequence contains a user's long-term, stable preferences, such as a preference for cost-effectiveness, quality, or style. For the generative retrieval paradigm, explicitly inputting short behavior sequences makes it easier for the model to predict which categories of items users are most likely to click on.

For an e-commerce search platform, the short behavior sequences include the user's latest entered queries Seq_{query} and clicked items Seq_{short} . We directly input the SIDs of these queries and items into the prompt, following the constructed user ID and the input query.

3.2.3 Implicit Long Behavior Sequence. For e-commerce platforms, user long behavior sequences primarily consist of three types: the click, the order, and the search relevant unit (RSU) [11] sequence, and the length of the behavior sequence is almost up to 10^3 . Therefore, it is almost impossible to integrate this information into the format of a handcrafted textual prompt. For each item within these sequences, we first map its keyword-enhanced embedding e_i^q to a corresponding semantic ID (*sid* for simplicity), and then get RQ clustering centroid representation through the lookup method, which is considered to contain different levels of semantic information. Furthermore, we aggregate the centroids according to different levels, which not only allows the GR to systematically learn the user preferences at different levels, but also saves a lot of resources.

The overall pipeline is shown below, each item in the long-term historical sequence is replaced by the features of its RQ clustering

centroid representative:

$$\begin{aligned} \text{Item}_{sid} &= \text{RQ}(e_i^o) \\ \text{Item}_{emb} &= \text{Emb_lookup}(\text{Item}_{sid}) \end{aligned}$$

For long-term historical sequence, overall behavior embedding is shown as

$$\begin{aligned} \mathbf{M}_{click} &= \left\{ \sum_{i=1}^{L1} \text{Item}_{emb}^1, \sum_{i=1}^{L2} \text{Item}_{emb}^2, \sum_{i=1}^{L3} \text{Item}_{emb}^3 \right\} \\ \mathbf{M}_{order} &= \left\{ \sum_{i=1}^{L1} \text{Item}_{emb}^1, \sum_{i=1}^{L2} \text{Item}_{emb}^2, \sum_{i=1}^{L3} \text{Item}_{emb}^3 \right\} \\ \mathbf{M}_{RSU} &= \left\{ \sum_{i=1}^{L1} \text{Item}_{emb}^1, \sum_{i=1}^{L2} \text{Item}_{emb}^2, \sum_{i=1}^{L3} \text{Item}_{emb}^3 \right\} \\ \mathbf{Q}^{(i)} &= \text{QFormer}(\mathbf{M}_{click}, \mathbf{M}_{order}, \mathbf{M}_{RSU}) \end{aligned} \quad (4)$$

where $\mathbf{M}_{click}$, $\mathbf{M}_{order}$, and $\mathbf{M}_{RSU}$ are referred to as click / order / RSU sequence item emb, and share the same size. To be specific, $\mathbf{M} \in \mathbb{R}^{N_M \times d_{model}}$ ($d_{model} = 768$).

These three user behavior sequence injection methods efficiently model users' short-term and long-term personalized preferences from different perspectives. Unlike the stacked behavior sequence concatenation used in MCA ranking stage, this modeling approach not only utilizes resources efficiently but also leverages the inference capabilities of GR models.

3.3 Unified Encoder-Decoder Architecture

In this section, we introduce the construction of OneSearch from the perspective of feature engineering. The input of OneSearch $\mathbf{X}_U$ consists of four parts: 1) User distinctive ID, denoted as uid , which is constructed as detailed in § 3.2. 2) Entered query q , as well as its SID SID_q ; 3) User Short Behavior Sequence, containing the historical search queries $Seq_q = \{q_1, q_2, \dots, q_n\}$, the short clicked item sequence $Seq_{short} = \{s_1, s_2, \dots, s_n\}$. 4) Implicit long behavior sequence, denoted as $Seq_{long}^{emb} = \{l_1, l_2, \dots, l_n\}$. 5) user profile information $\mathcal{U}$, which is the crowd portrait fitted by the platform. Then OneSearch directly outputs the corresponding item lists $\mathcal{I}$. OneSearch can adopt either encoder-decoder models (e.g. BART [19], mT5 [39]), or the decoder-only models (e.g. Qwen3 [40]) as the backbone $\mathcal{M}$. The inference flow can be formalized as:

$$\mathcal{I} := \mathcal{M}(uid, q, SID_q, Seq_q, Seq_{short}, Seq_{long}^{emb}, \mathcal{U}). \quad (5)$$

As illustrated in Figure 4, our model adheres to the transformer-based [35] architecture, comprising an encoder that models $\langle user, query, behavior sequence \rangle$ information, and a decoder dedicated to item generation. We adopted the encoder-decoder models for the real online deployment, as it is effective, have architecturally accelerated training and inference performance. For the unified training, we insert a start token $t_{[BOS]}$ and a ending token $t_{[EOS]}$ at the first and last place, as well as a separate token $t_{[SEP]}$ between adjacent elements to form the input to the encoder. The inference output of $\mathcal{M}$ is the SIDs, and it can be adjusted throughout constrained or unconstrained beam search. While constrained beam search guides output to valid SIDs, it increases the decoding complexity (inference time), and unconstrained search explores all sequences

without explicit rules. GRID [16] has shown a similar performance to these two streaming. We also conduct the testing in § 4.2.

3.4 Preference Aware Reward System

Compared to the sequence coherence in recommendation systems, the strong relevance constraints between queries and items in search engines pose greater challenges for online MCA, often addressed by an independent relevance module in the ranking stage. However, achieving a trade-off between relevance and ranking is a typical Pareto optimality problem. For GR models, it is necessary not only to achieve semantic alignment between SIDs and the textual descriptions of queries and items but also to directly generate items that meet query relevance constraints and user preferences based on historical behavior sequences. The items generated by beam search should naturally balance conversion and correlation. Therefore, we propose a preference aware reward system, which includes a multi-stage supervised fine-tuning (SFT) and an adaptive reward system, to enhance the model's personalized ranking capability. The overall training framework is depicted in Figure 4(f).

3.4.1 Multi-stage Supervised Fine-tuning. Considering that the basic architecture (e.g., BART, T5) is pretrained with a large text corpus, but the input queries and items in OneSearch are represented using SIDs, it is essential to first achieve semantic alignment between SIDs and their corresponding textual descriptions. Subsequently, the model should be instructed to generate the desired items that align with user intentions. To address this, we have designed a multi-stage supervised fine-tuning (SFT) procedure.

- (1) **Semantic Content Alignment:** We set three sub-tasks: (a) Take the query/item text into the prompt as inputs and output the corresponding SIDs. (b) Take the SID as input and generate the original query/item text. (c) Input the query/item text and output the corresponding category information. The first two tasks aim to align the SID and text content, while the category prediction ensures relevance.
- (2) **Co-occurrence Synchronization:** This stage includes mutual prediction between query and item, and the same task between query SID and item SID. Here, user characteristics are ignored, aiming to learn the intrinsic semantics and collaborative relationships between queries and items based on a large amount of online interactive corpus.
- (3) **User Personalization Modeling:** After the aforementioned two stages, we introduce user information into the final stage, which aligns with online inference. Specifically, we concatenate user ID (§ 3.2.1), query, SID_q , Seq_q , Seq_{short} (§ 3.2.2), and Seq_{long}^{emb} (§ 3.2.3) as input, with item SID as the training label, to instruct model to learn distinctive personalization.

It should be noted that sliding window data augmentation is applied to the short behavior sequence to guide the model in learning changes in user interests and preferences. The sliding window strategy generates a new segment of the sequence and its subsequent item as the prediction target at each step by sliding a window along the user's Seq_{short} [50]. To prevent the window from becoming too large, we limit the maximum window length. This means we can augment m samples for $Seq_{short} = \{s_1, s_2, \dots, s_m\}$, with the

first sample having no sequence, and the second having the sequence with only one item s_1 . This approach has been validated for achieving robust and high-performing sequential recommend and generative retrieval models [16, 50]. For e-commerce search, this also helps handle new users with limited search history by training on shorter subsequences. More details in § 4.2.

3.4.2 Adaptive Reward System. Unlike OneRec-V1 [47], which uses a weighted P-Score (Preference Score) of multiple objectives to train one reward model followed by Early Clipped GRPO to guide the model in learning user preferences, here we use real online user interactions as feedback signals. These interactions provide more accessible and hierarchical feedback information. While this approach shares similarities with the recently proposed OneRec-V2 [48], there are significant differences in training data sampling and training paradigms. We adopt adaptive-weighted reward signals [12] to construct training data and implement a user-behavior-guided hybrid ranking framework to achieve personalized preference ranking.

Adaptive-weighted Reward Signal. Following OneSug [12] we categorize user interactive behaviors in the search system into six distinct levels: (1) items purchased in search scenario, (2) items of the same category purchased in recommendation scenarios, (3) clicked items, (4) items exposed but not clicked, (5) unshow items in the same category, and (6) random items from other categories. We assign base reward weights as $\lambda = [2.0, 1.5, 1.0, 0.5, 0.2, 0.0]$ for each level respectively. Considering that items with higher CTR and CVR in recent days are more likely to be selected by users, we utilize these two metrics to construct adaptive-weighted rewards. However, CTR and CVR often suffer from biased estimation. For example, a newly released item that was exposed only once and then clicked would have CTR at 100%. Conversely, genuinely popular items are often exposed by Online MCA under various similar but suboptimal queries, resulting in lower CTR and CVR. Therefore, we calibrate these two metrics as follows:

$$Cnt_T = \log((Cnt_{pos} + 10) \cdot (Cnt_{clk} + 10) \cdot (Cnt_{order} + 10)) \quad (6)$$

thus:

$$Ctr_i = \frac{\log(Cnt_{clk} + 10)}{Cnt_T}, \quad Cvr_i = \frac{\log(Cnt_{order} + 10)}{\log(Cnt_{clk} + 10)}. \quad (7)$$

The weighted reward score is then defined as:

$$r(q, i) = 2\lambda \cdot \frac{Ctr_i \cdot Cvr_i}{Ctr_i + Cvr_i}. \quad (8)$$

For each positive sample i_{pos} and negative sample i_{neg} , the user preference difference rw_Δ is computed as:

$$rw_\Delta = \frac{1.0}{r(q, i_{pos}) - r(q, i_{neg})}, \quad (9)$$

where smaller rw_Δ values encourage the model to distinguish nuanced differences in user interactive behaviors.

Reward Model Training. As discussed in OneRec-V2, the reward model in OneRec-V1 employs restricted sampling from a small subset of users to approximate global behavior, potentially learning specific patterns or biases that do not yield actual improvements. However, we also diverge from the feedback-driven preference alignment proposed in OneRec-V2, as the adoption of GRPO and its variants (e.g., ECPO, GBPO) tends to introduce more irrelevant SIDs, and preference rewards require careful tuning for

e-commerce search. Here we design an intuitive and effective reward model based on the Search-based Interest Model (SIM [28]) with a three-tower architecture. Each tower is dedicated to learning specific objectives—CTR, CVR, and CTCVR [25]—using binary cross-entropy loss. The final preference score is computed as:

$$Rscore = \lambda_1 \cdot CTR + \lambda_2 \cdot CVR + \lambda_3 \cdot CTCVR + 10 \cdot \lambda_4 \cdot S_{Rel}, \quad (10)$$

where λ_i represents tuned weights (set to 1 in our experiments). To ensure that results generated by OneSearch meet relevance constraints, we additionally incorporate an offline-calculated relevance score S_{Rel} with an amplified weight ($10 \cdot \lambda_4$).

This reward model differs from the click prediction model in the ranking stage of online MCA in two key aspects: (1) Feature dimensionality: While the ranking model utilizes thousands of features, our reward model only takes user ID, entered query, user behavior sequence, and profile as input, matching OneSearch's input space. (2) Sampling strategy: We additionally include items from the same category clicked in recommendation scenarios as training samples, with labels (1,1,1) for purchased items and (1,0,0) for clicked items. For computational efficiency, the reward model can directly leverage the online MCA ranking model, as we only distill the ranking order rather than absolute scores.

Hybrid Ranking Framework. The alignment stage comprises two components. First, we collect entered queries from real search logs and use the reward model to rerank items output by the fine-tuned OneSearch. We then select samples where ranking changes occur for list-wise DPO training. Items that are clicked or advanced in position by the reward score serve as positive samples, while items pushed back in ranking and those at lower positions serve as negative samples. This allows us to gather one positive sample and multiple negative samples per query for training. The optimization objective can be described as follows:

$$\mathcal{L} = -\mathbb{E} \left[\log \sigma \left(\log \sum_{i_l \in \mathcal{I}_l} \exp(rw_\Delta \max(0, \hat{r}_\theta(x_u, i_w) - \hat{r}_\theta(x_u, i_l) - \delta)) \right) + \alpha \log \pi_\theta(i_w | x_u) \right], \quad (11)$$

where $\mathcal{I}_l$ denotes the set of negative samples, and $\hat{r}_\theta(x_u, i_w)$ and $\hat{r}_\theta(x_u, i_l)$ represent rewards implicitly defined by the language model π_θ and reference model π_{ref} :

$$\hat{r}_\theta(x_u, i_w | l) = \beta \log \frac{\pi_\theta(i_w | l | x_u)}{\pi_{ref}(i_w | l | x_u)}. \quad (12)$$

The term $\log \pi_\theta(i_w | x_u)$ represents the log-likelihood (NLL loss) from the SFT stage. Noted that by combining the list-wise preference alignment with log-likelihood prediction of preferred samples, we establish a novel hybrid paradigm for generative ranking.

This approach primarily trains the initial OneSearch model, as the reward model requires additional computational resources for online generation and scoring. Moreover, since the reward model is trained on user interaction data from the traditional search system, it inherently limits OneSearch's ability to exceed Online MCA's performance ceiling. Therefore, in the second phase, we train the model using pure user interactions. Specifically, we collect positive samples from the first three interactive levels and negative samples from the last three levels, continuing training with the same loss.

Table 4: The overall procedure of the preference aware reward system. It contains a three-stage supervised fine-tuning schema for semantic alignment, co-occurrence synchronization, and user personalization modeling, as well as an adaptive reward system for the personalized preference ranking.

Procedure	SFT Stage 1	SFT Stage 2	SFT Stage 3	RL Stage
Objective	Semantic alignment	$\langle q, i \rangle$ co-occurrence	User personalization	Preference Alignment
Component	$query \leftrightarrow SID$ $item \leftrightarrow SID$ $query/item \mapsto category$ $SID \mapsto category$	$query \leftrightarrow item$ $query_SID \leftrightarrow item_SID$	$\begin{bmatrix} uid \& q \\ SID_q \& Seq_q \\ Seq_{short} \\ Seq_{long}^{emb} \end{bmatrix} \mapsto item_SID$	$\begin{bmatrix} user \& query \\ seq. feat. \\ item_{win} \\ item_{lose} \end{bmatrix} \mapsto Rank\ Score$

In practice, we periodically perform the first RL training using reward model-generated samples. This ensures the model adheres to the online distribution and learns capabilities from the MCA ranking model, which is trained with thousands of features and more parameters. The second phase, preference learning based on user interaction data, is updated as close to streaming as possible. This design aims to overcome online distribution limitations and further leverage the inference capabilities of generative models.

4 Experiment

In this section, we conduct comprehensive evaluations on practical industry datasets offline and rigorous A/B online tests to verify the feasibility of OneSearch. Furthermore, we would explore some ablation studies to facilitate further research on a unified end2end generative model for online serving.

Datasets We extracted the highly reliable user interactive pairs from Kuaishou’s mall search platform between May 2025 and August 2025 to facilitate the supervised fine-tuning (SFT) and DPO. It contains about 1 billion PVs, and all the following offline and ablation experiments were conducted on the full or part of this data. The collections spanned 91 days, with the first 90 days used for model training and the last day used as the test set.

Evaluation Metrics Similar to OneSug, here we take into account the recall and ranking performance. We employed HitRate@K and Mean Reciprocal Ranking (MRR) as the evaluation metrics, which are widely used in search and recommendation systems. All data presented were the average values for all tests.

Baseline Methods To more accurately evaluate the performance of OneSearch, we compare it with the output results of the real online multi-stage cascading architecture (referred to as onlineMCA). Noted that, unlike OneSug, we do not construct an offline MCA system, as the combined models cannot accurately reflect the performance of the online system. Real online e-commerce platforms typically employ multiple recall mechanisms and complex ranking processes with thousands of feature combinations. Using only one model at each stage for simulation would result in an unfair comparison of offline performance.

Implementation Details We adopt Bart-B [19] as the base pre-trained model for the testing and online deployment, as it is an efficient model with optimized architectural acceleration, and has been online applied in many scenarios in Kuaishou. Due to commercial confidentiality, we do not disclose the total parameters of the online model here, but it is at least 100 times larger than

Bart. The beam search size is set to 512 here to strike a balance between generation quality and latency. The maximum window length is set to $n=5$. The batch size for SFT and DPO is set to 512 and 128, respectively, with the latter being smaller because the list-wise DPO training takes more samples as inputs. For RQ-OPQ, the number of codebook layers $C = 5$ (3 layers for RQ-Kmeans, and 2 layers for residual OPQ). The codebook size W of each layer is (4096,1024,512|256,256). Some of the hyperparameters will be discussed in the following ablation study. The multi-stage supervised training is conducted every week, RL with the reward system is conducted daily, and the hybrid preference alignment with user interaction data is updated as close to the stream as possible. Actually, RL with a reward system can also be trained every week, as we found it does not bring significant performance gains, except during the global shopping festivals (e.g., 11.11 and 6.18).

4.1 Offline Performance

We use the real onlineMCA system as the baseline. Specifically, we selected 30,000 pairs of data with click behavior and 30,000 pairs with order behavior from user search logs. We then calculated the Hitrate@350 and MRR@350 for the top 350 items with real user interactions. For a comprehensive evaluation, we also computed the metrics for the data output by pre-ranking stages, but without the final ranking. As shown in Table 5, we found that the pre-ranking stage tends to aggregate items with user interactions (resulting in higher recall but much lower MRR), whereas the ranking stage focuses on placing intent items higher in the list. This highlights the optimization objective collision across MCA stages, as the final ranking can only reorder the items output by the pre-ranking stage, ultimately limiting the potential of the final ranking.

We tested the RQ-Kmeans and KHQE tokenization with different configurations. As shown in Table 2, we found that higher codebook utilization rate (CUR) and independent coding rate (ICR) lead to better recall and ranking performance. Additionally, we tested the effects of adding core keyword enhancement, L3 balanced kmeans, and the adaptive reward system. All of these enhancements improved the metrics to varying degrees. Notably, the adaptive reward preference learning significantly improved the model’s ranking capability, with average improvements of 1.80% and 3.24% in HR@350 and MRR@350, respectively.

The final solution, described as RQ-OPQ (2/256) \+ Adaptive RS, means the model adopts the tokenization of RQ-Kmeans (4096-1024-512) followed by OPQ (256-256), and trained by the full preference

Table 5: Offline performances of our proposed method with onlineMCA on the industry dataset. The best results are in bold, and sub-optimal results are underlined in each column. The "w/o ranking" means "without ranking", and the "\+ keywords" means "add keywords optimizations"

Method	order (30k)		click (30k)	
	HR@350	MRR@350	HR@350	MRR@350
OnlineMCA	51.74%	19.26%	64.40%	16.89%
w/o ranking	75.75%	4.19%	80.23%	3.00%
OPQ (8/256)	19.43%	9.55%	22.57%	7.42%
(1024-1024-1024)	57.39%	9.12%	63.63%	7.46%
(2048-1024-512)	58.29%	10.79%	65.39%	8.86%
(4096-1024-256)	58.57%	11.21%	64.51%	9.24%
(4096-1024-512)	59.58%	14.29%	62.49%	11.82%
\+ keywords	62.38%	14.30%	66.14%	12.10%
\+ l3 balanced	63.16%	13.59%	68.26%	11.67%
\+ Adaptive RS	64.33%	<u>16.11%</u>	<u>68.94%</u>	<u>13.80%</u>
RQ-OPQ (2/256)	65.05%	15.33%	68.88%	12.90%
\+ Adaptive RS	66.46%	18.38%	71.06%	16.33%

Table 6: Ablation study of multi-view behavior sequence injection. Slid. Window means the sliding window strategy.

Method	order (30k)		click (30k)	
	HR@350	MRR@350	HR@350	MRR@350
OneSearch	66.46%	18.38%	71.06%	16.33%
w/o User SIDs	-0.94%	-0.37%	-1.72%	-0.36%
w/o Seq_{short}	-3.43%	-1.53%	-4.15%	-1.32%
w/o Seq_{long}^{emb}	-2.26%	-1.01%	-3.00%	-1.05%
w/o Slid.Window	-1.95%	-0.81%	-1.80%	-0.70%

aware reward system. This configuration achieved a much higher recall metric (66.46% vs. 51.74% for order) and comparable MRR performance (18.38% vs. 19.26% for order) compared to the onlineMCA. We believe this approach can ensure personalized ranking capability while maximizing the placement of items that match the search intent at the front of the list. This configuration would be called OneSearch in the following section for brevity.

4.2 Ablation Study

To further demonstrate the performance of the proposed method, we evaluated the effectiveness of 1) the multi-view behavior sequence injection schema, 2) RQ-OPQ tokenization, showing changes in CUR and ICR over time with the introduction of new items, and 3) different OPQ encoding tokenizations.

The first evaluation is of different behavior sequences. As shown in Table 6, "w/o User SIDs" replaces the sequence-constructed user IDs (presented in § 3.1) with Hashing User ID [31], resulting in an average decrease of 1.33% in HR@350 and 0.36% in MRR. This indicates that constructing User IDs using sequences more adequately represents user personalization compared to assigning a

Table 7: Ablation study of different OPQ tokenizations.

Method	order (30k)		click (30k)	
	HR@10	MRR@10	HR@10	MRR@10
RQ-OPQ (2/256)	28.42%	14.15%	33.69%	11.94%
*-OPQ (4/256)	-2.36%	-1.77%	-2.52%	-1.56%
*-OPQ (4*2/256)	-10.20%	-5.57%	-11.77%	-3.84%
*-OPQ (4*4/256)	-24.18%	-11.83%	-27.11%	-9.61%

unique ID without semantic and collaborative information. The hashing method has also been shown to have limited improvement in recommendation systems [16]. We also found that explicitly incorporating short behavior sequences in prompt text and implicitly including long behavior sequences into the model can significantly enhance model performance. Particularly, the short behavior sequence can bring an average increase of 3.79% in HR@350 and 1.43% in MRR@350. The sliding window augmentation on the short sequence is also validated to be effective in guiding the model to learn changes in user interests and preferences.

The items in a search system are constantly changing, especially during global shopping festivals, where poorly selling items are removed, and many new items in potentially high-demand categories are introduced. Consequently, a pre-calculated item SIDs pool with balanced k-means is continuously disrupted. Over time, more items would aggregate under the same SIDs. We conducted an experiment to verify whether this change is significant. We used all available items as of July 15 to construct two tokenizers (RQ-Kmeans and RQ-OPQ) and tested the trends in CUR and ICR as new items were added. As shown in Figure 6, the change in numerical values is minimal, even after the promotions on August 18. For example, RQ-Kmeans saw a 1.11% decrease in CUR, while RQ-OPQ only decreased by 0.43%. These results also validate the superiority of RQ-OPQ Tokenization.

We also examined the impact of different hierarchical quantization encodings on items in Figure 5. As shown in Table 7, we computed two metrics with the top 10 items for quick validation. RQ-OPQ (2/256) is the basic configuration, and RQ-OPQ (4/256) means the residual embedding is tokenized by OPQ (256-256-256-256). RQ-OPQ (4*2/256) means all embeddings (the cluster of three layers and the residual one) are tokenized with OPQ (2/256), then (4*4/256) indicates further quantization. We found that the basic RQ-OPQ (2/256) achieved the highest performance. (4/256) perform weakly with increased sequence length and decoding complexity. The other two configurations were almost entirely ineffective, which is similar to the balanced k-means operation on full layers in § 3.1, as the hierarchical features were not distinctly represented, leading to many items being aggregated under the same SID.

4.3 Online A/B Testing

To verify OneSearch's effectiveness in online applications, we compared it with onlineMCA in KuaiShou's mall search platform through rigorous online A/B tests. It takes the short query the user entered as input and directly outputs the item candidates, where an item with a higher score would be displayed more prominently. We trained

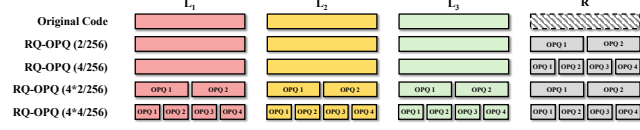


Figure 5: The different hierarchical quantization encodings of items.

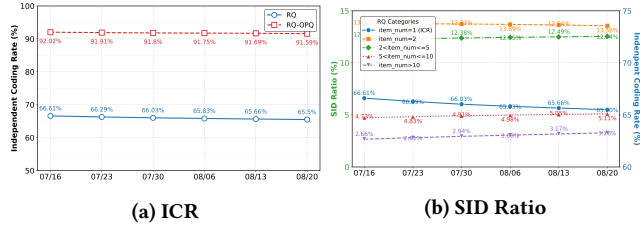


Figure 6: The ICR and SID ratio indicators of RQ-Kmeans after regular time intervals.

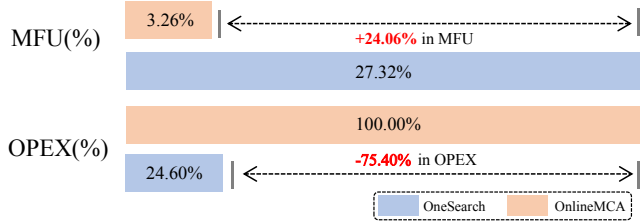


Figure 7: The comparisons of MFU and OPEX for onlineMCA and OneSearch.

two versions of the OneSearch model successively. *OneSearch*¹ refers to the model encoded using RQ-Kmeans and trained without incorporating the implicit long behavior sequence for modeling the user profile. *OneSearch*² is the model with all optimizations. Similar to OneRec-V1 [47], we established two experimental groups: one employing a pure generative model (*OneSearch*) and another reordering generative outputs with a reward model based selection (*OneSearch*_{RM}). As indicated in Table 8, the pure generative model with multi-stage supervised fine-tuning and adaptive preference learning can achieve comparable performance to the entire complex search system. By the introduction of RQ-OPQ and long behavior sequence, *OneSearch*² can confidently improve item CTR by 1.45%, PV CTR by 1.40%. Further applying reward model selection (*OneSearch*_{RM}²) achieved statistically significant improvements on all metrics, with 1.67% in item CTR, 3.14% in PV CTR, 1.78% in PV CVR, 2.40% in Buyer volume, and 3.22% in Order volume. For a clearer comparison, we perform additional experiments on the online search system, named MCA w/o ranking, which only uses the "recall and pre-ranking" module to predict the items, without the ranking stage. It significantly reduces all indicators, especially with 28.78% in Buyer, 39.14% in Order volume. This indirectly verifies that OneSearch has comparable ranking capabilities. These outstanding results show that OneSearch outperforms the onlineMCA,

Table 8: Online results for A/B testing. The black fonts indicate that the statistical significance (P-value) is smaller than 0.05, while the gray ones are larger than 0.05, which means the data are not yet confident.

Method	Item CTR	PV CTR	PV CVR	Buyer	Order
MCA w/o ranking	-9.97%	-20.33%	-11.55%	-28.78%	-39.14%
<i>OneSearch</i> ¹	-1.10%	-2.06%	+0.39%	+1.27%	-2.22%
<i>OneSearch</i> _{RM} ¹	+1.40%	+3.05%	+1.94%	+1.92%	+1.59%
<i>OneSearch</i> ²	+1.45%	+1.40%	-0.12%	-0.58%	-0.69%
<i>OneSearch</i> _{RM} ²	+1.67%	+3.14%	+1.78%	+2.40%	+3.22%

Table 9: Manual evaluation results for online experience.

Metric	Page Good Rate	Item Quality	Q-I Relevance
<i>OneSearch</i> ¹	0.84%	1.69%	1.40%
<i>OneSearch</i> ²	1.03%	2.12%	1.87%

and indicate it can update the complicated online system to a more balanced state without generating seesaw effects.

We also measured Model FLOPs Utilization (MFU) on flagship GPUs during serving inference. In Figure 7, the onlineMCA is only 3.26%, but OneSearch can achieve 27.32%, with a relative improvement of 700.38%. This significantly outperforms onlineMCA and the common large language models (LLMs), which typically reach 40% of MFU on H100 GPUs [7]. Furthermore, OneSearch significantly reduces communication and memory overhead, resulting in operational expenditure (OPEX) reduced to only 24.60% of the online search pipeline, promoting the application of GRs in search systems.

Last but not least, to ascertain the actual impacts on the online search experience, we conducted additional manual evaluations. We randomly selected 200 queries and extracted 3,200 query-item pairs from identical exposure positions, ensuring all other variables remained constant. We set three metrics as 1) page good rate - an evaluation indicator for the overall user experience, 2) item quality - Check whether the displayed products are counterfeit, have mismatched images and text, or have abnormal prices, and 3) query-item relevance - we engaged experts to rate each pair as "Good" (both subject and core keywords match), "Fair" (only subject matches), or "Bad" (subjects differ). The outcomes of these assessments are presented in Table 9. We can see that *OneSearch*² achieves substantial increases in page good rate by 1.03%, item quality by 2.12%, and query item relevance by 1.87%. The deployment of RQ-OPQ further enhances the relevance of model generation.

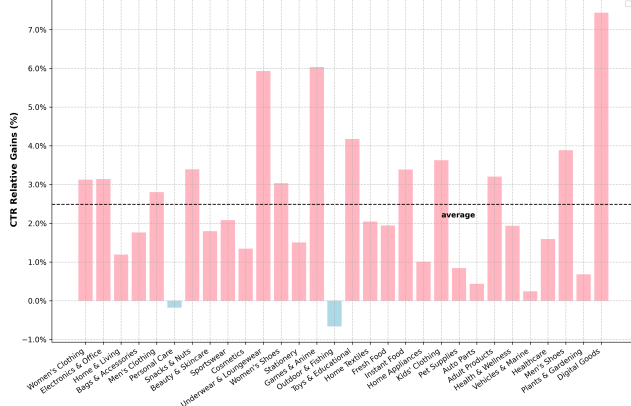
Ultimately, OneSearch has been successfully deployed for the entire traffic on the e-commerce detail page search engine in Kuaishou, 50% traffic on the mall search, and 20% traffic on the homepage e-commerce search platform for further investigation, which serves millions of users generating tens of millions of PVs daily.

4.4 Further Analysis

In this section, we mainly discuss three questions about the online deployment of the end2end generative framework and provide our investigations to facilitate further research.

Table 10: Online CTR gains for three query popularity.

Method	Top	Middle	Long-tail
OneSearch ²	+1.25%	+2.27%	+1.33%

**Figure 8: The online CTR relative gains for top 30 industries.**

1) **What are the main aspects of the online gains for the OneSearch model?** In our analysis, we focused on the dimensions of industry and query popularity. As illustrated in Figure 8, we calculated the CTR relative gains across the top 30 industries. Remarkably, 28 out of 30 industries experienced increases, with an average gain of 2.49%. These results were statistically significant, with P-values below 0.05. Although two industries showed negative effects, these were not statistically significant. Overall, the unified modeling optimization demonstrates substantial potential in addressing the inconsistent objectives of multi-stage processes in MCA systems, benefiting nearly all industries.

As for the query popularity dimension, we divided all prefixes into three categories: top (PV number daily larger than 1,000), middle (larger than 100 and less than 1,000), and long-tail (less than 100). The item CTR relative gains for each were listed in Table 10. Queries of all categories are enhanced with the OneSearch models. These results indicate that the rich semantic and interactive representations induced by keyword-enhanced hierarchical quantization encoding, multi-view behavior sequence, and the preference aware reward system can greatly improve the recognition of e-commerce search for queries of all popularity.

2) Does OneSearch have stronger reasoning capabilities?

In traditional e-commerce search scenarios, ranking models often involve thousands of features, and the combination of them can obscure some key attributes. Additionally, the structure of common ranking model typically consists of a simple stack of shallow neural networks, resulting in minimal reasoning capabilities. OneSearch, on the other hand, leverages users' long- and short-term sequential information to identify their potential interests and enhances the inference of user search intent through the attention mechanism

Table 11: Online CTR gains for cold-start items and users.

Object	Warm	Cold	Average
Item	+2.34%	+3.31%	+2.52%
User	+1.11%	+2.50%	+2.41%

of transformer structures. For instance, a female user who previously searched for "couple sneakers" and "Valentine's Day gifts" is likely seeking a pair of rings for both her partner and herself when searching for "silver ring." We observed in real logs that only OneSearch presented the relevant product, which was ultimately purchased by the user.

3) **How does OneSearch perform for cold-start users and items?** We conducted tests to evaluate the model's performance in cold-start scenarios. Here, we define cold items as those published within the last seven days with no interaction behavior, and cold users as those who have not used the Kuaishou app in the past 90 days. The specific comparison results are demonstrated in Table 11. Compared to the onlineMCA, we found that OneSearch's performance for cold-start items and users has improved by 3.31% and 2.50%, respectively. Both of which are greater than the metrics for warm ones. These results show that OneSearch can handle the cold-start issue well.

4) **What optimization points will OneSearch consider in the future?** The addition of OPQ-based tokenization can even quickly process new hotwords. We constructed a new keyword offline and added it to the textual descriptions of some items. Without reconstructing a new codebook, OneSearch was still able to generate SIDs for these items during inference. This finding further motivates us to consider online real-time encoding. We will explore in future research, aiming to achieve unified encoding and inference using a single generative model, thereby reducing the gap between scheduled encoding and streaming training phrase. Additionally, aligning user preferences through more robust reinforcement learning and incorporating multi-modal features (such as images and videos) for items can further enhance the reasoning capabilities.

5 Conclusion

In this paper, we present OneSearch, a pioneering end-to-end generative framework for e-commerce query search that effectively overcomes the limitations of traditional multi-stage cascading architecture. By employing a unified generative model, introducing the keyword-enhanced hierarchical quantization encoding, and injecting the multi-view behavior sequences, OneSearch achieves superior semantic understanding and personalization modeling. The preference aware reward strategy further refines the model's ability to capture user preferences, leading to improved ranking performance. Extensive offline and online evaluations confirm OneSearch's effectiveness in boosting query diversity, click-through rates, and business conversions. Its successful deployment on multiple Kuaishou search scenes underscores its practical applicability and potential to enhance industry revenue. OneSearch sets a new benchmark for industrial query search solutions, paving the way for future advancements in generative retrieval methods.

References

- [1] Alfred V. Aho and Margaret J. Corasick. 1975. Efficient string matching: an aid to bibliographic search. *Commun. ACM* 18, 6 (June 1975), 333–340. doi:10.1145/360825.360855
- [2] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. 2023. Qwen-VL: A Frontier Large Vision-Language Model with Versatile Abilities. *arXiv preprint arXiv:2308.12966* (2023).
- [3] Wei Bao, Hao Chen, Bang Lin, Tao Zhang, and Chengfu Huo. 2025. GRAIN: Group-Reinforced Adaptive Interaction Network for Cold-Start CTR Prediction in E-commerce Search. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2025, Padua, Italy, July 13-18, 2025*. ACM, 4275–4279. doi:10.1145/3726302.3731947
- [4] Junyi Chen, Lu Chi, Bingyue Peng, and Zehuan Yuan. 2024. HLLM: Enhancing Sequential Recommendations via Hierarchical Large Language Models for Item and User Modeling. arXiv:2409.12740 [cs.LR] <https://arxiv.org/abs/2409.12740>
- [5] Jiaxin Deng, Shiyao Wang, Kuo Cai, Lejian Ren, Qigen Hu, Weifeng Ding, Qiang Luo, and Guorui Zhou. 2025. OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment. arXiv:2502.18965 [cs.LR] <https://arxiv.org/abs/2502.18965>
- [6] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, et al. 2024. The Llama 3 Herd of Models. *CoRR* abs/2407.21783 (2024). doi:10.48550/ARXIV.2407.21783 arXiv:2407.21783
- [7] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv e-prints* (2024), arXiv:2407.
- [8] Ariel Evnine, Stratis Ioannidis, Dimitris Kalimeris, Shankar Kalyanaraman, Weiwei Li, Israel Nir, Wei Sun, and Udi Weinsberg. 2024. Achieving a Better Tradeoff in Multi-stage Recommender Systems through Personalization. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (Barcelona, Spain) (KDD '24)*. Association for Computing Machinery, New York, NY, USA, 4939–4950. doi:10.1145/3637528.3671593
- [9] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2014. Optimized Product Quantization. *IEEE Trans. Pattern Anal. Mach. Intell.* 36, 4 (2014), 744–755. doi:10.1109/TPAMI.2013.240
- [10] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: a factorization-machine based neural network for CTR prediction. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence (Melbourne, Australia) (IJCAI'17)*. AAAI Press, 1725–1731.
- [11] Tong Guo, Xuanping Li, Haitao Yang, Xiao Liang, Yong Yuan, Jingyou Hou, Bingqing Ke, Chao Zhang, Junlin He, Shunyu Zhang, et al. 2023. Query-dominant User Interest Network for Large-Scale Search Ranking. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 629–638.
- [12] Xian Guo, Ben Chen, Siyuan Wang, Ying Yang, Chenyi Lei, and et al. 2025. OneSug: The Unified End-to-End Generative Framework for E-commerce Query Suggestion. *CoRR* abs/2506.06913 (2025). doi:10.48550/ARXIV.2506.06913 arXiv:2506.06913
- [13] Yupeng Hou, Jiacheng Li, Ashley Shin, Jinsung Jeon, Abhishek Santhanam, Wei Shao, Kaveh Hassani, Ning Yao, and Julian McAuley. 2025. Generating Long Semantic IDs in Parallel for Recommendation. arXiv:2506.05781 [cs.LR] <https://arxiv.org/abs/2506.05781>
- [14] Jui-Ting Huang, Ashish Sharma, Shuying Sun, Li Xia, David Zhang, Philip Pronin, Janani Padmanabhan, Giuseppe Ottaviano, and Linjun Yang. 2020. Embedding-based Retrieval in Facebook Search. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '20)*. ACM, 2553–2561. doi:10.1145/3394486.3403305
- [15] Po-Sen Huang, Xiaodong He, Jianfeng Gao, Li Deng, Alex Acero, and Larry Heck. 2013. Learning deep structured semantic models for web search using clickthrough data. In *Proceedings of the 22nd ACM International Conference on Information & Knowledge Management (San Francisco, California, USA) (CIKM '13)*. Association for Computing Machinery, New York, NY, USA, 2333–2338. doi:10.1145/2505515.2505665
- [16] Clark Mingxuan Ju, Liam Collins, Leonardo Neves, Bhuvish Kumar, Louis Yufeng Wang, Tong Zhao, and Neil Shah. 2025. Generative Recommendation with Semantic IDs: A Practitioner's Handbook. arXiv:2507.22224 [cs.LR] <https://arxiv.org/abs/2507.22224>
- [17] Clark Mingxuan Ju, Liam Collins, Leonardo Neves, Bhuvish Kumar, Louis Yufeng Wang, Tong Zhao, and Neil Shah. 2025. Generative Recommendation with Semantic IDs: A Practitioner's Handbook. arXiv:2507.22224 [cs.LR] <https://arxiv.org/abs/2507.22224>
- [18] Doyup Lee, Chihoon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. 2022. Autoregressive Image Generation using Residual Quantization. arXiv:2203.01941 [cs.CV] <https://arxiv.org/abs/2203.01941>
- [19] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. 2019. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *arXiv preprint arXiv:1910.13461* (2019).
- [20] Jiacheng Li, Ming Wang, Jin Li, Jinmiao Fu, Xin Shen, Jingbo Shang, and Julian McAuley. 2023. Text Is All You Need: Learning Language Representations for Sequential Recommendation (KDD '23). Association for Computing Machinery, New York, NY, USA, 1258–1267. doi:10.1145/3580305.3599519
- [21] Mingming Li, Huimu Wang, Zuxu Chen, Guangtao Nie, Yiming Qiu, and et al. 2024. Generative Retrieval with Preference Optimization for E-commerce Search. arXiv:2407.19829 [cs.LR] <https://arxiv.org/abs/2407.19829>
- [22] Mingming Li, Huimu Wang, Zuxu Chen, Guangtao Nie, Yiming Qiu, Guoyu Tang, Lin Liu, and Jingwei Zhuo. 2024. Generative Retrieval with Preference Optimization for E-commerce Search. arXiv:2407.19829 [cs.LR] <https://arxiv.org/abs/2407.19829>
- [23] Zida Liang, Changfa Wu, Dunxian Huang, Weiqiang Sun, Ziyang Wang, and et al. 2025. TBGRRecall: A Generative Retrieval Model for E-commerce Recommendation Scenarios. arXiv:2508.11977 [cs.LR] <https://arxiv.org/abs/2508.11977>
- [24] Xinchun Luo, Jiangxia Cao, Tianyu Sun, Jinkai Yu, Rui Huang, Wei Yuan, Hezheng Lin, Yichen Zheng, Shiyao Wang, Qigen Hu, et al. 2024. Qarm: Quantitative alignment multi-modal recommendation at kuaishou. *arXiv preprint arXiv:2411.11739* (2024).
- [25] Xiao Ma, Liqin Zhao, Guan Huang, Zhi Wang, Zelin Hu, Xiaoqiang Zhu, and Kun Gai. 2018. Entire Space Multi-Task Model: An Effective Approach for Estimating Post-Click Conversion Rate. arXiv:1804.07931 [stat.ML] <https://arxiv.org/abs/1804.07931>
- [26] Fabian Mentzer, David Minnen, Eirikur Agustsson, and Michael Tschannen. 2023. Finite Scalar Quantization: VQ-VAE Made Simple. arXiv:2309.15505 [cs.CV] <https://arxiv.org/abs/2309.15505>
- [27] Ming Pang, Chunyuan Yuan, Xiaoyu He, Zheng Fang, Donghao Xie, and et al. 2025. Generative Retrieval and Alignment Model: A New Paradigm for E-commerce Retrieval. arXiv:2504.01403 [cs.LR] <https://arxiv.org/abs/2504.01403>
- [28] Pi Qi, Xiaoqiang Zhu, Guorui Zhou, Yujing Zhang, Zhe Wang, Lejian Ren, Ying Fan, and Kun Gai. 2020. Search-based User Interest Modeling with Lifelong Sequential Behavior Data for Click-Through Rate Prediction. arXiv:2006.05639 [cs.LR] <https://arxiv.org/abs/2006.05639>
- [29] Junyan Qiu, Ze Wang, Fan Zhang, Zuowu Zheng, Jile Zhu, and et al. 2025. One Model to Rank Them All: Unifying Online Advertising with End-to-End Learning. arXiv:2505.19755 [cs.LR] <https://arxiv.org/abs/2505.19755>
- [30] Yingqi Qu, Yuchen Ding, Jing Liu, Kai Liu, Ruiyang Ren, Wayne Xin Zhao, Daxiang Dong, Hua Wu, and Haifeng Wang. 2021. RocketQA: An Optimized Training Approach to Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, Online, 5835–5847. doi:10.18653/v1/2021.naacl-main.466
- [31] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, and et al. 2023. Recommender Systems with Generative Retrieval. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 10299–10315. https://proceedings.neurips.cc/paper_files/paper/2023/file/20dcab0f14046a5c6b02b61da9f13229-Paper-Conference.pdf
- [32] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. 2001. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th International Conference on World Wide Web (Hong Kong, Hong Kong) (WWW '01)*. Association for Computing Machinery, New York, NY, USA, 285–295. doi:10.1145/371920.372071
- [33] Yi Tay, Vinh Tran, Mostafa Dehghani, Jianmo Ni, Dara Bahri, and et al. 2022. Transformer Memory as a Differentiable Search Index. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 21831–21843. https://proceedings.neurips.cc/paper_files/paper/2022/file/892840a6123b5ec99ebaab8be1530fba-Paper-Conference.pdf
- [34] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. 2018. Neural Discrete Representation Learning. arXiv:1711.00937 [cs.LG] <https://arxiv.org/abs/1711.00937>
- [35] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [36] Ruoxi Wang, Rakesh Shivanna, Derek Zhiyuan Cheng, Sagar Jain, Dong Lin, Lichan Hong, and Ed H. Chi. 2021. DCN V2: Improved Deep & Cross Network and Practical Lessons for Web-scale Learning to Rank Systems. In *WWW '21: The Web Conference 2021, Virtual Event / Ljubljana, Slovenia, April 19-23, 2021*. ACM / IW3C2, 1785–1797. doi:10.1145/3442381.3450078
- [37] Zhipeng Wei, Kuo Cai, Junda She, Jie Chen, Minghao Chen, and et al. 2025. OneLoc: Geo-Aware Generative Recommender Systems for Local Life Service. (2025). arXiv:2508.14646 [cs.LR] <https://arxiv.org/abs/2508.14646>
- [38] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. C-Pack: Packaged Resources To Advance General Chinese Embedding. arXiv:2309.07597 [cs.CL]
- [39] Linting Xue, Noah Constant, Adam Roberts, Mihir Kale, Rami Al-Rfou, Aditya Siddhant, Aditya Barua, and Colin Raffel. 2020. mT5: A massively multilingual

- pre-trained text-to-text transformer. *arXiv preprint arXiv:2010.11934* (2020).
- [40] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
 - [41] Xiaoyong Yang, Yadong Zhu, Yi Zhang, Xiaobo Wang, and Quan Yuan. 2020. Large Scale Product Graph Construction for Recommendation in E-commerce. *arXiv:2010.05525* [cs.IR] <https://arxiv.org/abs/2010.05525>
 - [42] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, and et al. 2024. Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations. *arXiv:2402.17152* [cs.LG] <https://arxiv.org/abs/2402.17152>
 - [43] Luankang Zhang, Kenan Song, Yi Quan Lee, Wei Guo, Hao Wang, Yawen Li, Huifeng Guo, Yong Liu, Defu Lian, and Enhong Chen. 2025. Killing Two Birds with One Stone: Unifying Retrieval and Ranking with a Single Generative Recommendation Model. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Padua, Italy) (SIGIR '25). Association for Computing Machinery, New York, NY, USA, 2224–2234. doi:10.1145/3726302.3730017
 - [44] Zhixuan Zhang, Yuheng Huang, Dan Ou, Sen Li, Longbin Li, Qingwen Liu, and Xiaoyi Zeng. 2023. Rethinking the Role of Pre-ranking in Large-scale E-Commerce Searching System. *arXiv:2305.13647* [cs.IR] <https://arxiv.org/abs/2305.13647>
 - [45] Bowen Zheng, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, Ming Chen, and Ji-Rong Wen. 2024. Adapting Large Language Models by Integrating Collaborative Semantics for Recommendation. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. 1435–1448. doi:10.1109/ICDE60146.2024.00118
 - [46] Zuowu Zheng, Ze Wang, Fan Yang, Jiangke Fan, Teng Zhang, Yongkang Wang, and Xingxing Wang. 2025. EGA-V2: An End-to-end Generative Framework for Industrial Advertising. *arXiv:2505.17549* [cs.IR] <https://arxiv.org/abs/2505.17549>
 - [47] Guorui Zhou, Jiaxin Deng, Jinghao Zhang, Kuo Cai, Lejian Ren, and et al. 2025. OneRec Technical Report. *CoRR abs/2506.13695* (2025). doi:10.48550/ARXIV.2506.13695 *arXiv:2506.13695*
 - [48] Guorui Zhou, Hengrui Hu, Hongtao Cheng, Huanjie Wang, Jiaxin Deng, Jinghao Zhang, Kuo Cai, Lejian Ren, Lu Ren, Liao Yu, Pengfei Zheng, Qiang Luo, Qianqian Wang, Qigen Hu, Rui Huang, Ruiming Tang, Shiyao Wang, Shujie Yang, Tao Wu, Wuchao Li, Xinchun Luo, Xingmei Wang, Yi Su, Yunfan Wu, Zexuan Cheng, Zhanyu Liu, Zixing Zhang, Bin Zhang, Boxuan Wang, Chaoyi Ma, Chengru Song, Chenhui Wang, Chenglong Chu, Di Wang, Dongxue Meng, Dunju Zang, Fan Yang, Fangyu Zhang, Feng Jiang, Fuxing Zhang, Gang Wang, Guowang Zhang, Han Li, Honghui Bao, Hongyang Cao, Jiaming Huang, Jiapeng Chen, Jiaqiang Liu, Jinghui Jia, Kun Gai, Lantao Hu, Liang Zeng, Qiang Wang, Qidong Zhou, Rongzhou Zhang, Shengzhe Wang, Shihui He, Shuang Yang, Siyang Mao, Sui Huang, Tiantian He, Tingting Gao, Wei Yuan, Xiao Liang, Xiaoxiao Xu, Xugang Liu, Yan Wang, Yang Zhou, Yi Wang, Yiwu Liu, Yue Song, Yufei Zhang, Yunfeng Zhao, Zhixin Ling, and Ziming Li. 2025. OneRec-V2 Technical Report. *arXiv:2508.20900* [cs.IR] <https://arxiv.org/abs/2508.20900>
 - [49] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, and et al. 2018. Deep Interest Network for Click-Through Rate Prediction. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (London, United Kingdom) (KDD '18). Association for Computing Machinery, New York, NY, USA, 1059–1068. doi:10.1145/3219819.3219823
 - [50] Peilin Zhou, You-Liang Huang, Yueqi Xie, Jingqi Gao, Shoujin Wang, Jae Boum Kim, and Sunghun Kim. 2024. Is contrastive learning necessary? a study of data augmentation vs contrastive learning in sequential recommendation. In *Proceedings of the ACM Web Conference 2024*. 3854–3863.
 - [51] Jie Zhu, Zhifang Fan, Xiaoxie Zhu, Yuchen Jiang, and et al. 2025. RankMixer: Scaling Up Ranking Models in Industrial Recommenders. *arXiv:2507.15551* [cs.IR] <https://arxiv.org/abs/2507.15551>